

Cookies

Cookies are the underlying mechanism your Sessions guide was built on top of — this guide steps back to cover cookies as a broader HTTP concept: how they actually work at the protocol level, every attribute (not just the security-relevant ones), how scoping really works across domains and subdomains, and the browser storage alternatives that increasingly compete with cookies for holding sensitive data like tokens.

1. What a Cookie Actually Is

A cookie is a small piece of data a server asks the browser to store and automatically resend on future requests to that server. It's the core mechanism that gives HTTP — an inherently stateless protocol — any sense of continuity between requests at all.

```
Server → Browser: Set-Cookie: theme=dark
Browser → Server:  Cookie: theme=dark      (sent automatically on future requests)
```

Cookies aren't just for sessions — they're used for anything a site wants to persist across requests: session IDs, user preferences, tracking identifiers, A/B test group assignments, CSRF tokens, and more.

2. The **Set-Cookie** Header — Full Anatomy

```
Set-Cookie: sessionid=a8f5f167f44f4964; Domain=example.com; Path=/app;
Expires=Wed, 21 Oct 2026 07:28:00 GMT; Secure; HttpOnly; SameSite=Strict
```

Attribute	Purpose
name=value	The actual cookie data (only mandatory part)
Domain	Which domain(s) the cookie is sent to
Path	Which URL paths on that domain receive the cookie
Expires	Absolute expiration date/time
Max-Age	Expiration as a relative number of seconds (takes precedence over Expires when both are present)

Attribute	Purpose
<code>Secure</code>	Only sent over HTTPS
<code>HttpOnly</code>	Inaccessible to JavaScript
<code>SameSite</code>	Controls cross-site sending behavior

You already covered `Secure`, `HttpOnly`, and `SameSite` in depth in your Sessions guide — this guide fills in the scoping mechanics (`Domain` / `Path`) and lifecycle attributes those earlier notes only briefly mentioned, plus everything around how cookies interact with subdomains, third parties, and the browser storage alternatives.

3. Session Cookies vs. Persistent Cookies

```

Session cookie:  no Expires/Max-Age set at all
                  → deleted automatically when the browser is closed

Persistent cookie: Expires or Max-Age is set
                   → survives browser restarts, persists until that
                     date/duration passes (or is manually cleared)

```

Security implication worth remembering: a "remember me" feature (from your Authentication guide's Section 14) necessarily uses a persistent cookie — and a persistent cookie holding a session-equivalent token sitting on a shared/public computer is exactly the scenario your Sessions guide's idle-timeout discussion was warning about, just made concrete through this specific attribute.

4. Domain Scoping — Host-Only vs. Domain Cookies

This is one of the most commonly misunderstood parts of cookie behavior, and worth getting precisely right.

No `Domain` attribute set at all → "host-only" cookie:

```
Set-Cookie: session=abc123
```

Sent **only** to the exact host that set it — `app.example.com` — never to `example.com` itself, and never to any other subdomain like `blog.example.com`.

Domain attribute explicitly set:

```
Set-Cookie: session=abc123; Domain=example.com
```

Sent to `example.com` and every subdomain of it — `app.example.com`, `blog.example.com`, `mail.example.com`, all of them. Setting **Domain** always **broadens** scope; you can never use it to scope a cookie more narrowly than host-only.

Why this matters for security assessments: a session cookie unnecessarily scoped with `Domain=example.com` (instead of being left host-only to `app.example.com`) is now sent to *every* subdomain — including a forgotten, unpatched marketing microsite or a lower-security staging subdomain. If any one of those subdomains has an XSS vulnerability, that vulnerability can now steal a cookie meant for a completely different, more sensitive subdomain. This is a genuinely real, frequently-cited finding category called **"related-domain" or "cross-subdomain" cookie exposure**.

5. Path Scoping

```
Set-Cookie: admin_session=xyz; Path=/admin
```

The cookie is only sent on requests to `/admin` and anything beneath it (`/admin/users`, `/admin/settings`) — never on requests to `/blog` or the site root, even though it's the same domain.

Practical implication: **Path** scoping is a legitimate defense-in-depth tool (an admin-only session cookie scoped to `/admin` won't leak on a request to an unrelated, lower-security path on the same domain) but it's rarely relied upon as a primary control — **Domain** / **Path** scoping narrows *where the browser sends the cookie automatically*, but doesn't prevent a script running anywhere on that domain from reading a non-`HttpOnly` cookie via `document.cookie` regardless of the **Path** it was scoped to.

6. The **SameSite** Attribute — Deep Dive

Your Sessions guide introduced **SameSite** as a CSRF mitigation — worth expanding on its actual three values here, since the distinction matters a lot in practice:

Value	Behavior
Strict	Cookie is never sent on any cross-site request, including a user clicking a link from an external site into yours
Lax	Cookie is sent on top-level cross-site navigation via safe methods (GET , clicking a link) but not on cross-site POST requests, / <iframe> loads, or other subresource requests — this is the modern browser default when no SameSite value is explicitly set
None	Cookie is sent on all cross-site requests regardless of origin — requires the Secure attribute to also be set, or modern browsers reject it outright

Why Lax (the default) still leaves some CSRF risk: Lax blocks the classic auto-submitting cross-site POST form CSRF attack, but still allows the cookie on a simple GET request triggered by a top-level navigation — if an application performs a **state-changing action via GET** (a genuinely bad practice, but one that still exists in real apps), SameSite=Lax alone won't protect it.

When SameSite=None is legitimately necessary: cross-site embedded content that needs to carry authentication — a third-party widget, an embedded payment iframe, single sign-on flows spanning multiple domains. Any SameSite=None cookie deserves specific scrutiny in an assessment, since it's explicitly opting out of the browser's default cross-site protection.

7. Cookie Prefixes — A Lesser-Known Hardening Mechanism

Modern browsers support special cookie name prefixes that enforce certain attributes automatically, and reject the Set-Cookie header entirely if those requirements aren't met:

__Secure- prefix → the cookie MUST also have the Secure attribute set, or the browser refuses to set it at all

__Host- prefix → MUST have Secure set, MUST NOT have a Domain attribute (forcing host-only scope), and MUST have Path=/ — the strictest guarantee available

Set-Cookie: __Host-session=abc123; Secure; Path=/; HttpOnly; SameSite=Strict

The `__Host-` prefix is genuinely valuable specifically because it structurally **prevents** the cross-subdomain exposure problem from Section 4 — a cookie named with this prefix cannot be scoped broader than host-only, no matter what a developer accidentally sets `Domain` to, since the browser will simply refuse the cookie if `Domain` is present at all. Worth flagging the *absence* of these prefixes on sensitive session cookies as a minor hardening recommendation in a report, even when the underlying `Domain` / `Path` configuration is already reasonably tight.

8. Cookies vs. Other Browser Storage — `localStorage` and `sessionStorage`

This distinction comes up constantly once JavaScript-heavy frameworks (React/Angular/Vue apps, from your OWASP ZAP guide's AJAX Spider discussion) started storing auth tokens themselves rather than relying purely on server-set cookies.

	Cookies	<code>localStorage</code>	<code>sessionStorage</code>
Sent automatically with requests	Yes	No	No
Accessible to JavaScript	Yes, unless <code>HttpOnly</code>	Always	Always
Persistence	Configurable (session or persistent)	Until explicitly cleared	Until tab/window closes
Size limit	~4KB	~5-10MB	~5-10MB
CSRF-relevant	Yes (auto-sent)	No (must be manually attached to requests, e.g., an <code>Authorization</code> header)	No
XSS-relevant	Protected if <code>HttpOnly</code>	Always fully exposed to any XSS — no equivalent protection exists	Always fully exposed to any XSS

The genuinely important security trade-off here: storing a token in `localStorage` instead of an `HttpOnly` cookie completely eliminates CSRF risk for that token (since it's never auto-sent), but it also completely eliminates any protection against XSS-based theft — there is no `HttpOnly`-equivalent control for `localStorage`, full stop. This is an active, debated design trade-off in modern

web development (particularly around where to store JWTs for SPA/API-driven apps), and worth understanding as a trade-off rather than a simple "one is more secure" answer — the right choice genuinely depends on which threat (CSRF vs. XSS) the specific application is more exposed to and how well it defends against each.

9. Cookie Injection / Cookie Tossing (Cross-Subdomain Attack)

A more advanced attack technique worth knowing conceptually: since cookies scoped with a broad `Domain` attribute (Section 4) are shared across all subdomains, an attacker who controls *any* subdomain of a target (even a minor, seemingly unimportant one — a forgotten marketing microsite, a misconfigured S3 bucket serving under a subdomain) can potentially **set a cookie** that the browser will then send to the *legitimate, sensitive* subdomain.

```
Attacker controls: evil.example.com (via a takeover, subdomain
                    misconfiguration, or an XSS on that subdomain)

Attacker sets:      Set-Cookie: session=attacker-chosen-value; Domain=example.com

Victim's browser now sends THIS attacker-chosen session value to
app.example.com — the legitimate, sensitive application — since the
cookie is domain-scoped broadly enough to cover it
```

This is functionally a **session fixation attack (your Sessions guide, Section 6) delivered via a completely different mechanism** — instead of tricking a victim into clicking a link with a URL parameter, the attacker exploits the shared cookie jar across subdomains directly. It's exactly why subdomain takeover findings (your DNS/Subdomain Enumeration guide's `subjack` mention) are taken so seriously even when the taken-over subdomain itself seems low-value — it can become a launchpad for attacking a completely different, more important subdomain via cookie injection.

10. Practical Testing Workflow

1. Capture every Set-Cookie header across the application (proxy everything through ZAP/Burp per your OWASP ZAP guide)
2. For each cookie, check:

- Secure set? (especially for any session/auth cookie)
 - HttpOnly set? (unless the app has a specific, justified reason for JS to read it)
 - SameSite value - Strict/Lax appropriate, or an unjustified None?
 - Domain - host-only, or unnecessarily broadened? If broadened, is that genuinely required?
 - __Secure-/_Host- prefix used where appropriate?
 - Persistent cookie holding sensitive data with an excessively long expiration?
3. If the app also stores tokens in localStorage/sessionStorage, note this explicitly - it changes which threat model (CSRF vs XSS) matters most for that specific token, and should be reported as a design consideration, not automatically as a flaw
4. If ANY subdomain of the target is out of scope, compromised, or independently vulnerable, specifically test whether cookies are domain-scoped broadly enough for a cookie-tossing attack to be theoretically relevant - a good example of connecting a DNS/subdomain-enumeration-stage finding to a session-stage one

11. Prevention — Cookie Configuration Checklist

- Prefer host-only scope (omit Domain) unless cross-subdomain sharing is a genuine, deliberate requirement
- Use __Host- prefix on session cookies wherever browser support allows, to structurally enforce host-only + Secure + Path=/
 - Always set Secure and HttpOnly on any authentication-relevant cookie
 - Default to SameSite=Strict or Lax; reserve SameSite=None for genuinely necessary cross-site use cases, always paired with Secure
 - Scope Path as narrowly as the application's structure allows
 - Set explicit, reasonable expiration - avoid indefinite persistent cookies for anything sensitive
 - If storing tokens client-side outside of cookies, understand and document the resulting CSRF-vs-XSS trade-off explicitly rather than defaulting to it without consideration

12. Practical Lab Example

Using any app you've already proxied through ZAP (DVWA, WebGoat, or a PortSwigger Academy lab), pull up the full list of Set-Cookie headers issued across a normal browsing session and build a table matching Section 10's checklist for each one - this is a genuinely fast, five-minute exercise that surfaces real findings even on well-known lab applications, and it's exactly the kind of systematic

review that belongs in a VAPT report's session management section alongside the deeper session-behavior tests from your Sessions guide.

Kill Chain / ATT&CK / CWE — Full Mapping

Framework	Mapping
CWE	CWE-1004 (Sensitive Cookie Without HttpOnly Flag), CWE-614 (Sensitive Cookie in HTTPS Session Without Secure Attribute), CWE-1275 (Sensitive Cookie with Improper SameSite Attribute)
OWASP Top 10:2025	A02:2025 Security Misconfiguration (missing cookie attributes are a configuration failure) and A07:2025 Authentication Failures (when the affected cookie is session-related)
CVSS	Individually, a missing attribute is often Low/Medium on its own — severity climbs sharply once chained with XSS (HttpOnly missing) or CSRF (SameSite missing/None)
MITRE ATT&CK	T1539 (Steal Web Session Cookie) — the same technique from your Sessions guide, since cookies are the delivery mechanism for the session IDs discussed there

Quick Reference Cheat Sheet

```
Set-Cookie: name=value; Domain=...; Path=...; Expires=...; Max-Age=...;
           Secure; HttpOnly; SameSite=Strict|Lax|None
```

Domain scoping:

No Domain set → host-only (narrowest, safest default)

Domain=example.com → sent to ALL subdomains (broadens exposure)

SameSite:

Strict → never sent cross-site

Lax → sent on top-level GET navigation only (modern default)

None → sent everywhere, REQUIRES Secure

Hardening prefixes:

__Secure-name → forces Secure

__Host-name → forces Secure + host-only + Path=/'

Cookies vs storage:

Cookies → auto-sent (CSRF-relevant), HttpOnly protects vs XSS

localStorage/ → NOT auto-sent (CSRF-safe), but FULLY exposed to XSS

sessionStorage (no HttpOnly equivalent exists)